

2018-10-16: Analyzing libstagefright/MPEG4Extractor.cpp (v5.1.0_r1)

Wednesday, November 21, 2018 3:45 PM

16 October 2018

✕ Hafifot task to study StageFright: great idea!

Mr. Green tasked me with analyzing [libstagefright/MPEG4Extractor.cpp](#) as found in Lollipop v5.1.0 r1. The goal is to discover the vulnerabilities that existed then and which caused an uproar in the Android world.

First Steps

🔑 Need to download the identical Android framework code

My first step is to download the platform/frameworks/av for Lollipop v5.1.0 r1 source code. Having the source will allow the following:

- Use SourceInsight for analyzing the code
- Compile MPEG4Extractor.cpp using gcc to quickly find obvious vulnerabilities (e.g., integer conversions)
- Use cppcheck to find other potential vulnerabilities

Download the Android repository for platform/frameworks/av

I went to the page <https://android.googlesource.com/platform/frameworks/av/> as saw the following git command:
git clone <https://android.googlesource.com/platform/frameworks/av>

Executing the command downloaded the repository.

On the same web page, I drilled down, looking for version 5.1.0 r1:

[android-5.1.1 r2](#)
[android-5.0.2 r3](#)
[android-cts-5.1 r1](#)
[android-5.1.1 r1](#)
[android-5.1.0 r5](#)
[android-5.1.0 r3](#)
[android-5.1.0 r1](#)
[android-5.0.2 r1](#)
[android-wear-5.0.0 r1](#)
[android-5.0.1 r1](#)
[android-5.0.0 r7](#)

Clicking on the highlighted link shows:

[android](#) / [platform](#) / [frameworks](#) / [av](#) / **android-5.1.0_r1**

tag 7d2a64afea0be9842a26964b68ec858ce87d7510
tagger The Android Open Source Project <initial-contribution@android.com> Mon Mar 02 08:26:32 2015 -0800
object e2c8023444bf96739ff0e4c8aebc086bb73d3bf0

Android 5.1.0 release 1

[android](#) / [platform](#) / [frameworks](#) / [av](#) / **android-5.1.0_r1**

tag 7d2a64afea0be9842a26964b68ec858ce87d7510
tagger The Android Open Source Project <initial-contribution@android.com> Mon Mar 02 08:26:32 2015 -0800
object e2c8023444bf96739ff0e4cbaebc086bb73d3bf0

```
Android 5.1.0 release 1
-----BEGIN PGP SIGNATURE-----
Version: GnuPG v1

iEYEABECAAYFAIT0jzGACgkQ6K0/gZqxDnhFIQCfQ1YaPs8TTcC8CtdqxpV6Qzk
mpQAnjCRn4kmeUAaYYxmwrJlyRkk3Gy9
=moU3
-----END PGP SIGNATURE-----
```

commit e2c8023444bf96739ff0e4cbaebc086bb73d3bf0 [\[log\]](#) [\[tgz\]](#)
author Lajos Holnar <lajos@google.com> Tue Feb 10 16:06:38 2015 -0800
committer The Android Automerger <android-build@google.com> Wed Feb 11 14:34:02 2015 -0800
tree [d2f2b10f80b1ae99ca70ffc33d46e6f89f78c16e](#)
parent [f74acc7bb472b2bfa5f1aac368d598a760c29a77](#) [\[diff\]](#)

```
Revert "DO NOT MERGE: stagefright: add support for Intel's YUV420SP format in SoftwareRenderer"
This reverts commit 7a9518dcf637c01e5c953d77c7hd4489c80a828f.
```

What we want is the commit string, which will let us switch to this version. To switch to this commit, I used the command:

```
git checkout e2c8023444bf96739ff0e4cbaebc086bb73d3bf0
```

That's it, we have the exact version Mr. Green wanted me to examine.

17 October 2018



Android Xref is excellent way to examine a code base, use "Navigate" mode

Yesterday Mr. Green saw me working with printouts and Source Insight and asked if these are indeed more convenient for me. I realize now that I purposely did not download the entire Android Lollipop framework source code, which makes it difficult to examine low-level data structures. Also the printouts are thorough but slower and more cumbersome. I therefore agree with him that [Android Xref](#) might be the quickest way to get an overall picture of the code.

The following notes will be made while using Android Xref.

The background to analyzing the code is:

- The exploit begins from embedded media within an HTML page. When encountered, the browser instantiates an MPEG4Extractor object, causing the MPEG4Extractor constructor to be called. So we'll start by examining this constructor.
- We assume that we can craft the media object referred to within the HTML. The goal then is to craft the media file such that it causes a vulnerability to be caused within the Media server code.
- We'll find hypothetical vulnerabilities within the code. If found, we'll work backwards to craft a media file that causes it.

The constructor definition shows the following:

```
MPEG4Extractor::MPEG4Extractor(const sp<DataSource> &source)
: mMoofOffset(0),
  mDataSource(source),
  mInitCheck(NO_INIT),
  mHasVideo(false),
  mHeaderTimescale(0),
  mFirstTrack(NULL),
  mLastTrack(NULL),
  mFileMetaData(new MetaData),
  mFirstSINF(NULL),
  mIsDrm(false) {
}
```

So the data passed from the HTML is `sp<DataSource> &source`, which we can craft.

`sp` is the Android native template for 'strong pointer', what amounts to a safe pointer around `source`.

The `source` parameter is stored in the `mDataSource` member variable, incrementing the object reference count.

16:45

NOTE! Mr. Green sat with me to pass on methodological tips:



Perform two-pass scan through new code

- Since I'm examining `MPEG4Extractor.cpp`, I need to understand what MPEG4 is and its format (e.g., the concept of atoms).
- He recommends a two-pass scan through the code:
 - Pass 1
 - This is a "shallow" pass to get one's bearings in the code
 - Understand the structure of the code (e.g., what classes are there?)
 - While reading through the code, get a simple understanding of the variables
 - This is a good time to note integer mismatches
 - Be sure to record any thoughts you might have, for later use
 - Pass 2
 - Based on pass 1, you should have an idea of where to invest your time
 - Dive deeper into the code
 - Use the notes made in pass 1 to drill down

He pointed out to use the "Navigate" option in Android XRef for easily navigating to functions:



The "Android Xref" site is [mirrored here](#).

Notes from Pass 1 ("shallow pass")

It's not easy to find a clear and concise description of the MPEG-4 file format. For the official documentation, see [Information technology -- Coding of audio-visual objects -- Part 12: ISO base media file format](#).

Download the MP4 Inspector app for Windows ([from SourceForge](#)). It parses any MP4 file, revealing all atoms and their values.

The `MPEG4Extractor.cpp` code shows a list of MPEG-4 atoms:

0xa9ART	dinf	mehd	sinf	yrrc
0xa9alb	disk	meta	stbl	
0xa9day	edts	mfra	stco	
0xa9gen	elst	minf	stsc	
0xa9nam	enca	moof	stsd	
0xa9wrt	encv	moov	stss	
H263	esds	mp4a	stsz	
ID32	frma	mp4v	stts	
\xA9xyz	ftyp	mvex	stz2	
aART	gnre	mvhd	tenc	
albm	h263	name	tfhd	
auth	hdlr	perf	titl	
avc1	hev1	pssh	tkhd	

avcC	hvc1	s263	traf
co64	hvcC	saio	trak
covr	ilst	saiz	trex
cpil	mdat	samr	trkn
ctts	mdhd	sawb	trun
d263	mdia	schI	tx3g
data	mean	sidX	udta

An *atom* is a self-contained data unit that contains information about the video file in the form of a header and a data field. We can't dive into all of them, but let's try to understand a few.

Here's a table showing all atoms and their relative hierarchy:

Table 1 — Box types, structure, and cross-reference

ftyp					*	4.3	file type and compatibility	
pdin						8.43	progressive download information	
moov					*	8.1	container for all the metadata	
	mvhd				*	8.3	movie header, overall declarations	
	trak				*	8.4	container for an individual track or stream	
		tkhd			*	8.5	track header, overall information about the track	
		tref				8.6	track reference container	
		edts				8.25	edit list container	
			elst			8.26	an edit list	
		mdia			*	8.7	container for the media information in a track	
			mdhd		*	8.8	media header, overall information about the media	
			hdlr		*	8.9	handler, declares the media (handler) type	
			minf		*	8.10	media information container	
				vmhd		8.11.2	video media header, overall information (video track only)	
				smhd		8.11.3	sound media header, overall information (sound track only)	
				hmhd		8.11.4	hint media header, overall information (hint track only)	
				nmhd		8.11.5	Null media header, overall information (some tracks only)	
				dinf	*	8.12	data information box, container	
					dref	*	8.13	data reference box, declares source(s) of media data in track
				stbl	*	8.14	sample table box, container for the time/space map	
					stsd	*	8.16	sample descriptions (codec types, initialization etc.)
					stts	*	8.15.2	(decoding) time-to-sample
					ctts		8.15.3	(composition) time to sample
					stsc	*	8.18	sample-to-chunk, partial data-offset information
					stsz		8.17.2	sample sizes (framing)
					stz2		8.17.3	compact sample sizes (framing)
					stco	*	8.19	chunk offset, partial data-offset information
					co64		8.19	64-bit chunk offset
					stss		8.20	sync sample table (random access points)
					stsh		8.21	shadow sync sample table
					padb		8.23	sample padding bits
					stdp		8.22	sample degradation priority
					sdtg		8.40.2	independent and disposable samples
					sbgp		8.40.3.2	sample-to-group
					sgpd		8.40.3.3	sample group description
					subs		8.42	sub-sample information
	mvex					8.29	movie extends box	
		mehd				8.30	movie extends header box	
		trex			*	8.31	track extends defaults	
	ipmc					8.45.4	IPMP Control Box	
moof						8.32	movie fragment	
	mfhd				*	8.33	movie fragment header	
	traf					8.34	track fragment	
		tfhd			*	8.35	track fragment header	
		trun				8.36	track fragment run	
		sdtg				8.40.2	independent and disposable samples	
		sbgp				8.40.3.2	sample-to-group	
		subs				8.42	sub-sample information	
mfra						8.37	movie fragment random access	
	tfra					8.38	track fragment random access	
	mfro				*	8.39	movie fragment random access offset	
mdat						8.2	media data container	
free						8.24	free space	
skip						8.24	free space	
	udta					8.27	user-data	

ISO/IEC 14496-12:2005(E)

Table 1 (continued)

		cprt				8.28	copyright etc.
meta						8.44.1	metadata
	hdlr				*	8.9	handler, declares the metadata (handler) type
	dinf					8.12	data information box, container
		dref				8.13	data reference box, declares source(s) of metadata items
	ipmc					8.45.4	IPMP Control Box
	iloc					8.44.3	item location
	ipro					8.44.5	item protection
		sinf				8.45.1	protection scheme information box
			frma			8.45.2	original format box
			imif			8.45.3	IPMP Information box
			schm			8.45.5	scheme type box
			schI			8.45.6	scheme information box
						8.44.6	item information
	liinf						
	xml					8.44.2	XML container
	bxmI					8.44.2	binary XML container
	pitm					8.44.4	primary item reference

Atom Name	Description
moov	The <i>moov atom</i> , also referred to as the <i>movie atom</i> , defines the timescale, duration, display characteristics of the movie, as well as subatoms containing information for each track in the movie. The optimal location of the moov atom depends on the selected delivery method. A good description of the moov atom can be found here .

As we go through the code we'll refer back to the tables above.

MPEG4Extractor provides the following member functions:

```

MPEG4Extractor(const sp<DataSource> &source);
virtual ~MPEG4Extractor();
virtual size_t countTracks();
virtual sp<MediaSource> getTrack(size_t index);
virtual sp<MetaData> getTrackMetaData(size_t index, uint32_t flags);
virtual sp<MetaData> getMetaData();
virtual uint32_t flags() const;
// for DRM
virtual char* getDrmTrackInfo(size_t trackID, int *len);
status_t readMetaData();
status_t parseChunk(off64_t *offset, int depth);
status_t parseiTunesMetaData(off64_t offset, size_t size);
status_t parse3GPPMetaData(off64_t offset, size_t size, int depth);
void parseID3v2MetaData(off64_t offset);
status_t updateAudioTrackInfoFromESDS_MPEG4Audio(const void *esds_data, size_t esds_size);
status_t parseDrmSINF(off64_t *offset, off64_t data_offset);
status_t parseTrackHeader(off64_t data_offset, off64_t data_size);
status_t parseSegmentIndex(off64_t data_offset, size_t data_size);
Track *findTrackByMimePrefix(const char *mimePrefix);
MPEG4Extractor(const MPEG4Extractor &);
MPEG4Extractor &operator=(const MPEG4Extractor &);

```

Let's examine the main ones.

```
IMPEG4Extractor &operator=(const IMPEG4Extractor &),
```

Let's examine the main ones.

MPEG4Extractor(const sp<DataSource> &source)

```
MPEG4Extractor::MPEG4Extractor(const sp<DataSource> &source)
    : mMoofOffset(0),
      mDataSource(source),
      mInitCheck(NO_INIT),
      mHasVideo(false),
      mHeaderTimescale(0),
      mFirstTrack(NULL),
      mLastTrack(NULL),
      mFileMetaData(new MetaData),
      mFirstSINF(NULL),
      mIsDrm(false) {}
```

The main point is that the user-provided source parameter is stored in the mDataSource member variable. This will increment the object's reference count. All other variables are given default initial values.

MPEG-4 Definitions

ISO/IEC 14496-12:2005(E)

3 Definitions

For the purposes of this International Standard, the following terms and definitions apply.

3.1

Box:

An object-oriented building block defined by a unique type identifier and length (called 'atom' in some specifications, including the first definition of MP4).

3.2

Chunk:

A contiguous set of samples for one track.

3.3

Container Box:

A box whose sole purpose is to contain and group a set of related boxes.

3.4

Hint Track:

A special track which does not contain media data. Instead it contains instructions for packaging one or more tracks into a streaming channel.

3.5

Hint:

A tool that is run on a file containing only media, to add one or more hint tracks to the file and so facilitate streaming.

3.6

Movie Box:

A container box whose sub-boxes define the metadata for a presentation ('moov').

3.7

Media Data Box:

A container box which can hold the actual media data for a presentation ('mdat').

3.8

ISO Base Media File:

The name of the file format described in this specification.

3.9

Presentation:

One or more motion sequences (q.v.), possibly combined with audio.

3.10

Sample:

In non-hint tracks, a sample is an individual frame of video, a time-contiguous series of video frames, or a

one or more motion sequences (q.v.), possibly combined with audio.

3.10

Sample:

In non-hint tracks, a sample is an individual frame of video, a time-contiguous series of video frames, or a time-contiguous compressed section of audio. In hint tracks, a sample defines the formation of one or more streaming packets. No two samples within a track may share the same time-stamp.

3.11

Sample Description:

A structure which defines and describes the format of some number of samples in a track.

ISO/IEC 14496-12:2005(E)

3.12

Sample Table:

A packed directory for the timing and physical layout of the samples in a track.

3.13

Track:

A collection of related samples (q.v.) in an ISO base media file. For media data, a track corresponds to a sequence of images or sampled audio. For hint tracks, a track corresponds to a streaming channel.

Looking at the MP4 Inspector output, I see the data within the MPEG-4 file is big-endian.

Note that in a random 'box' ('moov' in this case) the first two 32-bit integers denote the 'chunk size' and 'chunk type':

Item	Value
Box Information	
Type	moov
Description	Movie Box
Size	4,221 bytes
Header Size	8 bytes
Data Size	4,213 bytes
Begin Position	1,051,515 (0x100B7B)
End Position	1,055,735 (0x101BF7)
Path	\moov
Children Boxes	
mvhd	Movie Header Box
trak	Track Box
trak	Track Box
udta	User Data Box

	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	
00100B7B	00	00	10	7D	6D	6F	6F	76	00	00	00	6C	6D	76	68	64	...}moov...lmvhd
00100B8B	00	00	00	00	7C	25	B0	80	CF	F0	58	31	00	00	03	E8	 %.I0x1...è
00100B9B	00	00	14	C0	00	01	00	00	01	00	00	00	00	00	00	00	...À.....
00100BAB	00	00	00	00	00	01	00	00	00	00	00	00	00	00	00	00	
00100BBB	00	00	00	00	00	01	00	00	00	00	00	00	00	00	00	00	
00100BCB	00	00	00	00	40	00	00	00	00	00	00	00	00	00	00	00	@.....
00100BDB	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00100BEB	00	00	00	03	00	00	06	45	74	72	61	6B	00	00	00	5C	Etrak...\
00100BFB	74	6B	68	64	00	00	00	0F	7C	25	B0	80	7C	25	B0	80	tkhd.... %. %.
00100CAF	00	00	00	01	00	00	00	00	00	00	14	00	00	00	00	00	

00100B0B	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
00100BEB	00 00 00 03 00 00 06 45 74 72 61 6B 00 00 00 5C	Etrak...\
00100BFB	74 6B 68 64 00 00 00 0F 7C 25 B0 80 7C 25 B0 80	tkhd.... %. %. .
00100C0B	00 00 00 01 00 00 00 00 00 00 14 A0 00 00 00 00	
00100C1B	00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00	
00100C2B	00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00	

The chunk type is 'moov', while the chunk size is 0x107D = 4,221 (big-endian). This size includes the two 32-bit integers. The data size is equal to 4221 - 8 = 4,213. The following code performs this calculation, although it is obtuse:

```
829     off64_t chunk_data_size = *offset + chunk_size - data_offset;
```

Clearer would have been:

```
off64_t chunk_data_size = chunk_size - (data_offset - *offset);
```

Function `parseChunk` makes a recursive call to itself:

```
831     if (chunk_type != FOURCC('c', 'p', 'r', 't')
832         && chunk_type != FOURCC('c', 'o', 'v', 'r')
833         && mPath.size() == 5 && underMetaDataPath(mPath)) {
834         off64_t stop_offset = *offset + chunk_size;
835         *offset = data_offset;
836         while (*offset < stop_offset) {
837             status_t err = parseChunk(offset, depth + 1);
838             if (err != OK) {
839                 return err;
840             }
841         }
```

Here's a suspicious section of code:

```
904         off64_t stop_offset = *offset + chunk_size;
905         *offset = data_offset;
906         while (*offset < stop_offset) {
907             status_t err = parseChunk(offset, depth + 1);
908             if (err != OK) {
909                 return err;
910             }
911         }
```

In line 904, `*offset` is a signed long long int, while `chunk_size` is `uint64_t`. According to the rule, their sum should be converted to unsigned int, then assigned to `stop_offset`, a signed integer. This means we can set `stop_offset` to a negative value. The following while loop will therefore never end.

Wrong! If `stop_offset` is negative the while loop will never be entered.

OK, I see that `parseChunk()` calls itself recursively. One can craft a `DataSource` buffer with atoms that can create an atom tree of any size. Can we read past the inputted buffer? Can we cause a buffer overflow of the buffer within the object?

HTML media element as attack vector, Javascript probes properties and return values

I had an interesting conversation with Mr. Green. My question to him was: supposing I find a vulnerability, how can information be returned to the attacker. He explained as follows:

- The attack vector is an HTML media element, which specifies a source file containing our specially-crafted MPEG-4 container containing the meta data.
- We also assume there is Javascript code **that can probe and call methods and examine properties belonging to the element**

The attack vector is an HTML meta element, which specifies a source file containing our specially crafted meta data container containing the meta data.

- We also assume there is Javascript code **that can probe and call methods and examine properties belonging to the element**.
- If done correctly, the meta data will cause a vulnerability to occur within MPEG4Extractor.cpp.
- The hope is that the vulnerability will overwrite or modify something that can be detected and read by the Javascript mentioned above.

As an example, he gave the following: suppose the crafted MPEG-4 file caused the data at an address of our choice to be placed in the width or height attributes. If we could do this, the Javascript code could iterate over a range of addresses, returning a different 32 bits of data each time. This would be a perfect info leak.

His example used a heap buffer overwrite as the attack method. If two objects are adjacent in heap memory, overwriting the lower addressed one could modify something of value in the higher-addressed one.

I had a question: when reading source code, we have no idea what structures or objects are adjacent in memory, so how can we perform such an analysis?

 First look for vulnerability, only then look to exploit it

NOTE! His answer is very important to internalize:

- In the first stage the researcher looks for a **vulnerability** in the source code (or disassembled code). Only when one is found can the researcher move to the second stage.
- In the second stage, the researcher now needs to study the vulnerability in a live environment, precisely to see what valuable effect it can have. Hopefully this will lead to an **exploit** (i.e., “proof of concept”).

18 October 2018

I returned to examine the MPEG4Extractor.cpp listing, and again feel that parseChunk is the place to focus my efforts. The code is complicated and long enough, and it does the actual reading of data, to warrant a closer look.

NOTE! I realize that the core of any vulnerability will be calls to memcpy and memset, so it would be smart to locate all such calls in the code and work backwards from there. Do the functions containing these function calls have immediate vulnerabilities?

 Look at all memcpy/memset/strcpy calls for vulnerabilities

Here are the functions that call memcpy / memset.

```
ssize_t MPEG4DataSource::readAt(off64_t offset, void *data, size_t size) {
    Mutex::Autolock autoLock(mLock);
    if (offset >= mCachedOffset
        && offset + size <= mCachedOffset + mCachedSize) {
        memcpy(data, &mCache[offset - mCachedOffset], size);
        return size;
    }
    return mSource->readAt(offset, data, size);
}

status_t MPEG4Extractor::readMetaData() {
    [. . .]
    // copy pssh data into file metadata
    int psshsize = 0;
    for (size_t i = 0; i < mPssh.size(); i++) {
        psshsize += 20 + mPssh[i].datalen;
    }
    if (psshsize) {
        char *buf = (char*)malloc(psshsize);
        char *ptr = buf;
        for (size_t i = 0; i < mPssh.size(); i++) {
            memcpy(ptr, mPssh[i].uuid, 20); // uuid + length
```

```

    char *buf = (char *)malloc(psshsize);
    char *ptr = buf;
    for (size_t i = 0; i < mPssh.size(); i++) {
        memcpy(ptr, mPssh[i].uuid, 20); // uuid + length
        memcpy(ptr + 20, mPssh[i].data, mPssh[i].datalen);
        ptr += (20 + mPssh[i].datalen);
    }
    mFileMetaData->setData(kKeyPssh, 'pssh', buf, psshsize);
    free(buf);
}
return mInitCheck;
}
status_t MPEG4Extractor::parseChunk(off64_t *offset, int depth) {
    [...]
    case FOURCC('t', 'x', '3', 'g'):
    {
        uint32_t type;
        const void *data;
        size_t size = 0;
        if (!mLastTrack->meta->findData(
            kKeyTextFormatData, &type, &data, &size)) {
            size = 0;
        }
        uint8_t *buffer = new (std::nothrow) uint8_t[size + chunk_size];
        if (buffer == NULL) {
            return ERROR_MALFORMED;
        }
        if (size > 0) {
            memcpy(buffer, data, size);
        }
        if ((size_t)(mDataSource->readAt(*offset, buffer + size, chunk_size))
            < chunk_size) {
            delete[] buffer;
            buffer = NULL;
            // advance read pointer so we don't end up reading this again
            *offset += chunk_size;
            return ERROR_IO;
        }
        mLastTrack->meta->setData(
            kKeyTextFormatData, 0, buffer, size + chunk_size);
        delete[] buffer;
        *offset += chunk_size;
        break;
    }
    [...]
    return OK;
}
MPEG4Source::MPEG4Source(
    const sp<MPEG4Extractor> &owner,
    const sp<MetaData> &format,
    const sp<DataSource> &dataSource,
    int32_t timeScale,
    const sp<SampleTable> &sampleTable,
    Vector<SidxEntry> &sidx,
    const Trex *trex,
    off64_t firstMoofOffset)
: mOwner(owner),
  [...]
  mSrcBuffer(NULL) {
    memset(&mTrackFragmentHeaderInfo, 0, sizeof(mTrackFragmentHeaderInfo));
    mFormat->findInt32(kKeyCryptoMode, &mCryptoMode);
    mDefaultIVSize = 0;
    mFormat->findInt32(kKeyCryptoDefaultIVSize, &mDefaultIVSize);
    uint32_t keytype;
    const void *key;
    size_t keysize;
    if (mFormat->findData(kKeyCryptoKey, &keytype, &key, &keysize)) {
        CHECK(keysize <= 16);
    }
}

```

```

    const void *key,
    size_t keysize;
    if (mFormat->findData(kKeyCryptoKey, &keytype, &key, &keysizes)) {
        CHECK(keysize <= 16);
        memset(mCryptoKey, 0, 16);
        memcpy(mCryptoKey, key, keysize);
    }
    [...]
}

status_t MPEG4Source::read(
    MediaBuffer **out, const ReadOptions *options) {
    [...]
    if ((!mIsAVC && !mIsHEVC) || mWantsNALFragments) {
        [...]
    } else {
        [...]
        if (usesDRM) {
            CHECK(mBuffer != NULL);
            mBuffer->set_range(0, size);
        } else {
            uint8_t *dstData = (uint8_t *)mBuffer->data();
            size_t srcOffset = 0;
            size_t dstOffset = 0;
            while (srcOffset < size) {
                bool isMalFormed = (srcOffset + mNALLengthSize > size);
                size_t nalLength = 0;
                if (!isMalFormed) {
                    nalLength = parseNALSize(&mSrcBuffer[srcOffset]);
                    srcOffset += mNALLengthSize;
                    isMalFormed = srcOffset + nalLength > size;
                }
                if (isMalFormed) {
                    ALOGE("Video is malformed");
                    mBuffer->release();
                    mBuffer = NULL;
                    return ERROR_MALFORMED;
                }
                if (nalLength == 0) {
                    continue;
                }
                CHECK(dstOffset + 4 <= mBuffer->size());
                dstData[dstOffset++] = 0;
                dstData[dstOffset++] = 0;
                dstData[dstOffset++] = 0;
                dstData[dstOffset++] = 1;
                memcpy(&dstData[dstOffset], &mSrcBuffer[srcOffset], nalLength);
                srcOffset += nalLength;
                dstOffset += nalLength;
            }
            CHECK_EQ(srcOffset, size);
            CHECK(mBuffer != NULL);
            mBuffer->set_range(0, dstOffset);
        }
        [...]
    }
}

status_t MPEG4Source::fragmentedRead(
    MediaBuffer **out, const ReadOptions *options) {
    [...]
    if ((!mIsAVC && !mIsHEVC) || mWantsNALFragments) {
        [...]
    } else {
        [...]
        if (usesDRM) {
            CHECK(mBuffer != NULL);
            mBuffer->set_range(0, size);
        } else {
            uint8_t *dstData = (uint8_t *)mBuffer->data();

```

```

        CHECK(mBuffer != NULL);
        mBuffer->set_range(0, size);
    } else {
        uint8_t *dstData = (uint8_t *)mBuffer->data();
        size_t srcOffset = 0;
        size_t dstOffset = 0;
        while (srcOffset < size) {
            bool isMalFormed = (srcOffset + mNALLengthSize > size);
            size_t nalLength = 0;
            if (!isMalFormed) {
                nalLength = parseNALSize(&mSrcBuffer[srcOffset]);
                srcOffset += mNALLengthSize;
                isMalFormed = srcOffset + nalLength > size;
            }
            if (isMalFormed) {
                ALOGE("Video is malformed");
                mBuffer->release();
                mBuffer = NULL;
                return ERROR_MALFORMED;
            }
            if (nalLength == 0) {
                continue;
            }
            CHECK(dstOffset + 4 <= mBuffer->size());
            dstData[dstOffset++] = 0;
            dstData[dstOffset++] = 0;
            dstData[dstOffset++] = 0;
            dstData[dstOffset++] = 1;
            memcpy(&dstData[dstOffset], &mSrcBuffer[srcOffset], nalLength);
            srcOffset += nalLength;
            dstOffset += nalLength;
        }
        CHECK_EQ(srcOffset, size);
        CHECK(mBuffer != NULL);
        mBuffer->set_range(0, dstOffset);
    }

    [...]
}

```

It was not clear to me how the readAt() calls work (i.e., what the calling hierarchy is). I now see that MPEG4Extractor() will typically receive a DataSource denoting, say, a FileSource:

```

ssize_t FileSource::readAt(off64_t offset, void *data, size_t size) {
    if (mFd < 0) {
        return NO_INIT;
    }
    Mutex::Autolock autoLock(mLock);
    if (mLength >= 0) {
        if (offset >= mLength) {
            return 0; // read beyond EOF.
        }
        int64_t numAvailable = mLength - offset;
        if ((int64_t)size > numAvailable) {
            size = numAvailable;
        }
    }
    if (mDecryptHandle != NULL && DecryptApiType::CONTAINER_BASED
        == mDecryptHandle->decryptApiType) {
        return readAtDRM(offset, data, size);
    } else {
        off64_t result = lseek64(mFd, offset + mOffset, SEEK_SET);
        if (result == -1) {
            ALOGE("seek to %lld failed", offset + mOffset);
            return UNKNOWN_ERROR;
        }
        return ::read(mFd, data, size);
    }
}

```

```

        ALOGE("seek to %lld failed", offset + mOffset);
        return UNKNOWN_ERROR;
    }
    return ::read(mFd, data, size);
}
}

```

Here we can see what checks, if any, are performed when `readAt()` is called.

A New Line of Thinking

Here's a new line of thinking I have.

`MPEG4Extractor` supports internal caching. This first occurs when the parser encounters an `stbl` atom / box:

```

switch(chunk_type) {
case FOURCC('m', 'o', 'o', 'v'):
case FOURCC('t', 'r', 'a', 'k'):
case FOURCC('m', 'd', 'i', 'a'):
case FOURCC('m', 'i', 'n', 'f'):
case FOURCC('d', 'i', 'n', 'f'):
case FOURCC('s', 't', 'b', 'l'):
case FOURCC('m', 'v', 'e', 'x'):
case FOURCC('m', 'o', 'o', 'f'):
case FOURCC('t', 'r', 'a', 'f'):
case FOURCC('m', 'f', 'r', 'a'):
case FOURCC('u', 'd', 't', 'a'):
case FOURCC('i', 'l', 's', 't'):
case FOURCC('s', 'i', 'n', 'f'):
case FOURCC('s', 'c', 'h', 'i'):
case FOURCC('e', 'd', 't', 's'):
{
    if (chunk_type == FOURCC('s', 't', 'b', 'l')) {
        ALOGV("sampleTable chunk is %" PRIu64 " bytes long.", chunk_size);
        if (mDataSource->flags()
            & (DataSource::kWantsPrefetching
                | DataSource::kIsCachingDataSource)) {
            sp<MPEG4DataSource> cachedSource =
                new MPEG4DataSource(mDataSource);
            if (cachedSource->setCachedRange(*offset, chunk_size) == OK) {
                mDataSource = cachedSource;
            }
        }
    }
}
[...]
```

The cache is instantiated by a call to `SetCachedRange()`:

```

status_t MPEG4DataSource::setCachedRange(off64_t offset, size_t size) {
    Mutex::Autolock autoLock(mLock);
    clearCache();
    mCache = (uint8_t *)malloc(size);
    if (mCache == NULL) {
        return -ENOMEM;
    }
    mCachedOffset = offset;
    mCachedSize = size;
    ssize_t err = mSource->readAt(mCachedOffset, mCache, mCachedSize);
    if (err < (ssize_t)size) {
        clearCache();
        return ERROR_IO;
    }
    return OK;
}

```

```

    }
    return OK;
}

```

I believe the cache's purpose is to cache the stbl data only. Any called for stbl data will hit the cache. Otherwise the readAt() call is passed up to the original DataSource object passed to the constructor.

```

ssize_t MPEG4DataSource::readAt(off64_t offset, void *data, size_t size) {
    Mutex::Autolock autoLock(mLock);
    if (offset >= mCachedOffset
        && offset + size <= mCachedOffset + mCachedSize) {
        memcpy(data, &mCache[offset - mCachedOffset], size);
        return size;
    }
    return mSource->readAt(offset, data, size);
}

```

Which sub-boxes access data within the cached stbl buffer? The following tables shows who they are:

			stbl	*	8.14	sample table box, container for the time/space map
			stsd	*	8.16	sample descriptions (codec types, initialization etc.)
			stts	*	8.15.2	(decoding) time-to-sample
			ctts		8.15.3	(composition) time to sample
			stsc	*	8.18	sample-to-chunk, partial data-offset information
			stsz		8.17.2	sample sizes (framing)
			stz2		8.17.3	compact sample sizes (framing)
			stco	*	8.19	chunk offset, partial data-offset information
			co64		8.19	64-bit chunk offset
			stss		8.20	sync sample table (random access points)
			stsh		8.21	shadow sync sample table
			padb		8.23	sample padding bits
			stdp		8.22	sample degradation priority
			sdtc		8.40.2	independent and disposable samples
			sbgp		8.40.3.2	sample-to-group
			sgpd		8.40.3.3	sample group description
			subs		8.42	sub-sample information

The idea is to pass in an (offset, size) pair, both which are signed integers, to pass the following if statement. We don't control the value of offset, but we can control size by crafting the MPEG-4 container of any of the stbl sub-boxes (e.g., stsd).

```

off64_t offset;
size_t size;
off64_t mCachedOffset;
size_t mCachedSize;

```

Suppose we cause size to be very large (i.e., close to MAX_UINT). If we do, memcpy will copy too much data to data.

```

if (offset >= mCachedOffset
    && offset + size <= mCachedOffset + mCachedSize) {
    memcpy(data, &mCache[offset - mCachedOffset], size);
    return size;
}

```

Here is what the stds processing code looks like:

Here is what the stds processing code looks like:

```
case FOURCC('s', 't', 's', 'd'):
{
    if (chunk_data_size < 8) {
        return ERROR_MALFORMED;
    }
    uint8_t buffer[8];
    if (chunk_data_size < (off64_t)sizeof(buffer)) {
        return ERROR_MALFORMED;
    }
    if (mDataSource->readAt(
        data_offset, buffer, 8) < 8) {
        return ERROR_IO;
    }
    if (U32_AT(buffer) != 0) {
        // Should be version 0, flags 0.
        return ERROR_MALFORMED;
    }
    uint32_t entry_count = U32_AT(&buffer[4]);
    if (entry_count > 1) {
        // For 3GPP timed text, there could be multiple tx3g boxes contain
        // multiple text display formats. These formats will be used to
        // display the timed text.
        // For encrypted files, there may also be more than one entry.
        const char *mime;
        CHECK(mLastTrack->meta->findCString(kKeyMIMETYPE, &mime));
        if (strcascmp(mime, MEDIA_MIMETYPE_TEXT_3GPP) &&
            strcascmp(mime, "application/octet-stream")) {
            // For now we only support a single type of media per track.
            mLastTrack->skipTrack = true;
            *offset += chunk_size;
            break;
        }
    }
    off64_t stop_offset = *offset + chunk_size;
    *offset = data_offset + 8;
    for (uint32_t i = 0; i < entry_count; ++i) {
        status_t err = parseChunk(offset, depth + 1);
        if (err != OK) {
            return err;
        }
    }
    if (*offset != stop_offset) {
        return ERROR_MALFORMED;
    }
    break;
}
```

When I showed it to Mr. Green he found the flaw in the 'vulnerability'. The vulnerability rested on the ability to insert a large box size (e.g., 0xFFFFFFFF) into the MPEG-4 meta data which would overflow the if expression. The reason this will not occur is found at the beginning of parseChunk:

```
status_t MPEG4Extractor::parseChunk(off64_t *offset, int depth) {
    ALOGV("entering parseChunk %lld/%d", *offset, depth);
    uint32_t hdr[2];
    if (mDataSource->readAt(*offset, hdr, 8) < 8) {
        return ERROR_IO;
    }
    uint64_t chunk_size = ntohl(hdr[0]);
    uint32_t chunk_type = ntohl(hdr[1]);
```

The chunk size is a 4-byte 32-bit integer, read into `hdr[]` via the `readAt()` call. However, the value is then copied into

```
uint32_t chunk_type = htonl(hdr[1]);
```

The chunk size is a 4-byte 32-bit integer, read into `hdr[]` via the `readAt()` call. However, the value is then copied into `chunk_size`, **which is an unsigned 64-bit integer**. This causes **zero** extension (not **sign** extension!) resulting in a 64-bit unsigned integer of `0x00000000FFFFFFFF`. This is too small to cause an unsigned 64-bit integer overflow in the `if` statement.

Oh well, we tried!



Onward and upward, looking for other vulnerabilities.

21 October 2018

This morning I wanted to compile `MPEG4Extractor.cpp` using `gcc -c` to see if it finds the StageFright vulnerability. Immediately I encountered problems with missing header files. Some of these header files are very generic (e.g. `Errors.h`) and would require my downloading **everything**.



Method to compile code that needs many include files

NOTE! In the end I used the following method to copy multiple include folders into the WSL environment:

- (a) Create a folder called `myinclude` somewhere in the project
- (b) Compile the C/C++ file with `gcc`

```
gcc -c -I../../include -Imyinclude MPEG4Extractor.cpp
```

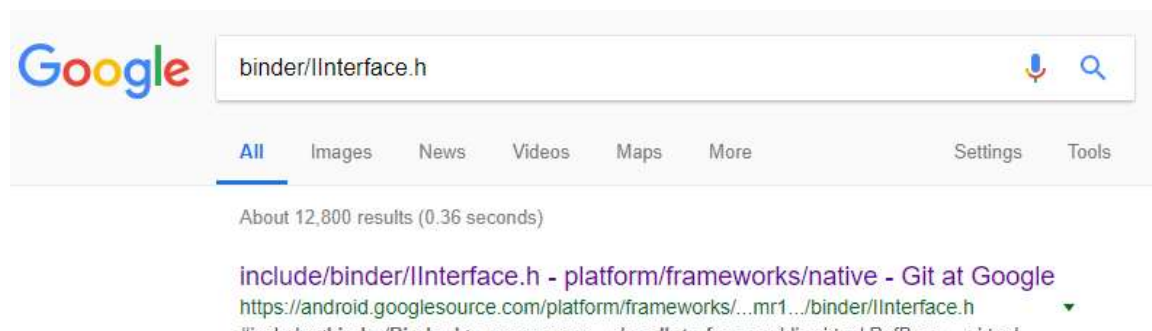
- (c) Note any missing header files

```
brown@DESKTOP-AE2I7DM:~/dev/learning/analyzing-stagefright/playground/av/media/libstagefright$ gcc -c -I../../include -Imyinclude MPEG4Extractor.cpp
In file included from ../../include/media/stagefright/DataSource.h:29,
                 from include/MPEG4Extractor.h:23,
                 from MPEG4Extractor.cpp:28:
../../include/drm/DrmManagerClient.h:21:10: fatal error: binder/IInterface.h: No such
file or directory
#include <binder/IInterface.h>
      ^~~~~~
compilation terminated.
brown@DESKTOP-AE2I7DM:~/dev/learning/analyzing-stagefright/playground/av/media/libstagefright$
```

So we're missing at least one header file from the `binder` folder found somewhere.

- (d) Locate the missing folder in Android's Git repository

Search for `binder/IInterface.h` in Google Search, note the one pointing to `android.googlesource.com`:



include/binder/IIInterface.h - platform/frameworks/native - Git at Google

<https://android.googlesource.com/platform/frameworks/native/+/jb-mr1.1-dev/include/binder/IIInterface.h>

```
#include <binder/Binder.h>. namespace ... class IIInterface : public virtual RefBase ... virtual  
sp<IIInterface> queryLocalInterface(const String16& _descriptor);
```

(e) Open the page:

[android](#) / [platform](#) / [frameworks](#) / [native](#) / [jb-mr1.1-dev](#) / . / [include](#) / [binder](#) / **IIInterface.h**

blob: 5f9f69c042bf9ff1bb69acb83bd4f140f28fcbd2 [file] [log] [blame]

```
1  /*  
2   * Copyright (C) 2005 The Android Open Source Project  
3   *  
4   * Licensed under the Apache License, Version 2.0 (the "License");  
5   * you may not use this file except in compliance with the License.  
6   * You may obtain a copy of the License at
```

(f) Click on the parent folder link (e.g., binder) to see the entire include folder contents

[android](#) / [platform](#) / [frameworks](#) / [native](#) / [jb-mr1.1-dev](#) / . / [include](#) / **binder**

tree: d5f1adfed74bfa95d05fd2b18a5a64d27353dce9 [path history] [tgz]

-  [Binder.h](#)
-  [BinderService.h](#)
-  [BpBinder.h](#)
-  [IBinder.h](#)
-  [IIInterface.h](#)
-  [IMemory.h](#)
-  [IPCThreadState.h](#)
-  [IPermissionController.h](#)
-  [IServiceManager.h](#)
-  [MemoryBase.h](#)
-  [MemoryDealer.h](#)
-  [MemoryHeapBase.h](#)
-  [Parcel.h](#)
-  [PermissionCache.h](#)
-  [ProcessState.h](#)

(g) Right-click on the [tgz] link to get the URL for downloading a tar.gz file containing all of the header files.

(h) In WSL, create a folder with the same parent folder name, all this under the myinclude folder.

(i) CD into the newly-created folder.

(j) Download the TGZ file by issuing the WGET command in the clipboard:

WGET <https://android.googlesource.com/platform/frameworks/native/+/archive/jb-mr1.1-dev/include/binder.tar.gz>

(k) Uncompress the file:

```
tar -xzf binder.tar.gz
```

(l) Delete the *.gz file, return to step (a).

(l) Delete the *.gz file, return to step (a).

Compile code with maximal warnings to find vulnerability

Now that we have all the required header files, we can compile, asking gcc to display suspicious code warnings. The gcc command used in the end was:

```
gcc -c -I../../include -Imyinclude -Wno-multichar MPEG4Extractor.cpp
```

The compilation output was:

```
brown@DESKTOP-AE2I7DM:~/dev/learning/analyzing-stagefright/playground/av/media/libstagefright$
gcc -c -I../../include -Imyinclude -Wno-multichar MPEG4Extractor.cpp
In file included from myinclude/cutils/log.h:40,
    from myinclude/utils/Log.h:31,
    from myinclude/utils/Vector.h:24,
    from myinclude/utils/SortedVector.h:24,
    from myinclude/utils/KeyedVector.h:24,
    from ../../include/media/stagefright/DataSource.h:25,
    from include/MPEG4Extractor.h:23,
    from MPEG4Extractor.cpp:28:
myinclude/cutils/uio.h:33:8: error: redefinition of 'struct iovec'
    struct iovec {
        ^~~~~~
In file included from /usr/include/x86_64-linux-gnu/sys/socket.h:26,
    from /usr/include/netinet/in.h:23,
    from /usr/include/arpa/inet.h:22,
    from include/MPEG4Extractor.h:21,
    from MPEG4Extractor.cpp:28:
/usr/include/x86_64-linux-gnu/bits/types/struct_iovec.h:26:8: note: previous definition of
'struct iovec'
    struct iovec
        ^~~~~~
In file included from myinclude/utils/SortedVector.h:24,
    from myinclude/utils/KeyedVector.h:24,
    from ../../include/media/stagefright/DataSource.h:25,
    from include/MPEG4Extractor.h:23,
    from MPEG4Extractor.cpp:28:
myinclude/utils/Vector.h: In member function 'const android::Vector<T>&
android::Vector<TYPE>::operator=(const android::Vector<TYPE>&) const':
myinclude/utils/Vector.h:245:64: error: no matching function for call to
'android::Vector<TYPE>::operator=(const android::VectorImpl&) const'
    VectorImpl::operator = (static_cast<const VectorImpl&>(rhs));
                                   ^
In file included from myinclude/utils/Vector.h:25,
    from myinclude/utils/SortedVector.h:24,
    from myinclude/utils/KeyedVector.h:24,
    from ../../include/media/stagefright/DataSource.h:25,
    from include/MPEG4Extractor.h:23,
    from MPEG4Extractor.cpp:28:
myinclude/utils/VectorImpl.h:56:29: note: candidate: 'android::VectorImpl&
android::VectorImpl::operator=(const android::VectorImpl&)' <near match>
    VectorImpl&      operator = (const VectorImpl& rhs);
        ^~~~~~
myinclude/utils/VectorImpl.h:56:29: note:   passing 'const android::Vector<TYPE>*' as 'this'
argument discards qualifiers
In file included from myinclude/utils/String16.h:22,
    from myinclude/binder/IBinder.h:22,
    from myinclude/binder/Binder.h:20,
    from myinclude/binder/IInterface.h:21,
    from ../../include/drm/DrmManagerClient.h:21,
    from ../../include/media/stagefright/DataSource.h:29,
    from include/MPEG4Extractor.h:23,
    from MPEG4Extractor.cpp:28:
```

```

        from ../../include/drm/DrmManagerClient.h:21,
        from ../../include/media/stagefright/DataSource.h:29,
        from include/MPEG4Extractor.h:23,
        from MPEG4Extractor.cpp:28:
myinclude/Utils/Unicode.h: At global scope:
myinclude/Utils/Unicode.h:25:18: error: redeclaration of C++ built-in type 'char32_t' [-fpermissive]
    typedef uint32_t char32_t;
                    ^~~~~~
myinclude/Utils/Unicode.h:26:18: error: redeclaration of C++ built-in type 'char16_t' [-fpermissive]
    typedef uint16_t char16_t;
                    ^~~~~~
In file included from include/MPEG4Extractor.h:25,
                 from MPEG4Extractor.cpp:28:
MPEG4Extractor.cpp: In member function 'android::status_t
android::MPEG4Extractor::parseChunk(off64_t*, int)':
../../include/media/stagefright/Utils.h:31:9: error: left operand of shift expression '(-87
<< 24)' is negative [-fpermissive]
    (c1 << 24 | c2 << 16 | c3 << 8 | c4)
    ^~~~~~
MPEG4Extractor.cpp:1540:14: note: in expansion of macro 'FOURCC'
    case FOURCC('\xA9', 'x', 'y', 'z'):
    ^~~~~~
brown@DESKTOP-AE2I7DM:~/dev/learning/analyzing-stagefright/playground/av/media/libstagefright$

```

Nothing pops up as an obvious vulnerability.

 Compile code with CppCheck tool to find vulnerability

NOTE! Checking with CppCheck

Using a similar command line as above, we can check MPEG4Extractor.cpp using CppCheck:

```

cppcheck --enable=all --includes-file=../../include --includes-file=myinclude
MPEG4Extractor.cpp

```

Here is the output:

```

brown@DESKTOP-AE2I7DM:~/dev/learning/analyzing-stagefright/playground/av/media/libstagefright$ cppcheck --
enable=all --includes-file=../../include --includes-file=myinclude MPEG4Extractor.cpp
Checking MPEG4Extractor.cpp ...
[MPEG4Extractor.cpp:2341]: (warning) %x in format string (no. 1) requires 'unsigned int *' but the argument type
is 'signed int *'.
[MPEG4Extractor.cpp:2341]: (warning) %x in format string (no. 2) requires 'unsigned int *' but the argument type
is 'signed int *'.
[MPEG4Extractor.cpp:925] -> [MPEG4Extractor.cpp:928]: (warning) Either the condition 'cur' is redundant or there
is possible null pointer dereference: cur.
[MPEG4Extractor.cpp:676]: (style) The scope of the variable 'dataLen' can be reduced.
[MPEG4Extractor.cpp:2453]: (style) The scope of the variable 'tmp' can be reduced.
[MPEG4Extractor.cpp:671]: (style) Variable 'classSize' is assigned a value that is never used.
[MPEG4Extractor.cpp:2154]: (style) Variable 'ctime' is assigned a value that is never used.
[MPEG4Extractor.cpp:2155]: (style) Variable 'mtime' is assigned a value that is never used.
[MPEG4Extractor.cpp:2157]: (style) Variable 'duration' is assigned a value that is never used.
[MPEG4Extractor.cpp:2167]: (style) Variable 'dx' is assigned a value that is never used.
[MPEG4Extractor.cpp:2170]: (style) Variable 'dy' is assigned a value that is never used.
[MPEG4Extractor.cpp:2862]: (style) Variable 'ElementInstanceTag' is assigned a value that is never used.
[MPEG4Extractor.cpp:2863]: (style) Variable 'Profile' is assigned a value that is never used.
[MPEG4Extractor.cpp:2864]: (style) Variable 'SamplingFrequencyIndex' is assigned a value that is never used.
[MPEG4Extractor.cpp:2869]: (style) Variable 'NumAssocDataElements' is assigned a value that is never used.
[MPEG4Extractor.cpp:2870]: (style) Variable 'NumValidCccElements' is assigned a value that is never used.
[MPEG4Extractor.cpp:2874]: (style) Variable 'MonoMixdownElementNumber' is assigned a value that is never used.
[MPEG4Extractor.cpp:2879]: (style) Variable 'StereoMixdownElementNumber' is assigned a value that is never used.
[MPEG4Extractor.cpp:2884]: (style) Variable 'MatrixMixdownIndex' is assigned a value that is never used.
[MPEG4Extractor.cpp:2885]: (style) Variable 'PseudoSurroundEnable' is assigned a value that is never used.
[MPEG4Extractor.cpp:2891]: (style) Variable 'FrontElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:2897]: (style) Variable 'SideElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:2903]: (style) Variable 'BackElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:2909]: (style) Variable 'LfeElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:3040] -> [MPEG4Extractor.cpp:61]: (warning) Virtual function 'stop' is called from
destructor 'MPEG4Source' at line 3040

```

```
[MPEG4Extractor.cpp:2897]: (style) Variable 'SideElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:2903]: (style) Variable 'BackElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:2909]: (style) Variable 'LfeElementTagSelect' is assigned a value that is never used.
[MPEG4Extractor.cpp:3040] -> [MPEG4Extractor.cpp:61]: (warning) Virtual function 'stop' is called from
destructor '~MPEG4Source()' at line 3040.
[include/MPEG4Extractor.h:53]: (style) Class 'MPEG4Extractor' has a constructor with 1 argument that is not
explicit.
[include/SampleTable.h:35]: (style) Class 'SampleTable' has a constructor with 1 argument that is not explicit.
[MPEG4Extractor.cpp:170]: (style) Struct 'MPEG4DataSource' has a constructor with 1 argument that is not
explicit.
[MPEG4Extractor.cpp:3270]: (error) Common realloc mistake: 'mCurrentSampleInfoSizes' nulled but not freed upon
failure
[MPEG4Extractor.cpp:3301]: (error) Common realloc mistake: 'mCurrentSampleInfoOffsets' nulled but not freed upon
failure
[MPEG4Extractor.cpp:4404]: (style) The function 'SniffMPEG4' is never used.
[MPEG4Extractor.cpp:405]: (style) The function 'countTracks' is never used.
[MPEG4Extractor.cpp:4219]: (style) The function 'findTrackByMimePrefix' is never used.
[MPEG4Extractor.cpp:554]: (style) The function 'getDrmTrackInfo' is never used.
[MPEG4Extractor.cpp:3636]: (style) The function 'getFormat' is never used.
[MPEG4Extractor.cpp:396]: (style) The function 'getMetaData' is never used.
[MPEG4Extractor.cpp:2567]: (style) The function 'getTrack' is never used.
[MPEG4Extractor.cpp:423]: (style) The function 'getTrackMetaData' is never used.
[MPEG4Extractor.cpp:268]: (style) The function 'hexdump' is never used.
[MPEG4Extractor.cpp:3661]: (style) The function 'read' is never used.
[MPEG4Extractor.cpp:3046]: (style) The function 'start' is never used.
(information) Cppcheck cannot find all the include files (use --check-config for details)
brown@DESKTOP-AE2I7DM:~/dev/learning/analyzing-stagefright/playground/av/media/libstagefright$cppcheck
Removing unimportant warnings/errors:
[MPEG4Extractor.cpp:3040] -> [MPEG4Extractor.cpp:61]: (warning) Virtual function 'stop' is called from
destructor '~MPEG4Source()' at line 3040.
[include/MPEG4Extractor.h:53]: (style) Class 'MPEG4Extractor' has a constructor with 1 argument that is not
explicit.
[MPEG4Extractor.cpp:3270]: (error) Common realloc mistake: 'mCurrentSampleInfoSizes' nulled but not freed upon
failure
[MPEG4Extractor.cpp:3301]: (error) Common realloc mistake: 'mCurrentSampleInfoOffsets' nulled but not freed upon
failure
```

CppCheck does not show any obvious vulnerabilities.

References for MPEG

- [Atomic Parsley](#)
 - Important information about MPEG-4 in general, and iTunes in particular.
- [Information technology — Coding of audio-visual objects — Part 12: ISO base media file format](#)

Continuing Analysis

PSSH = “Protection System Specific Header”

The Final Frontier

At some point, I told Mr. Green that I did not find the vulnerability. He then pointed me to the code where the vulnerability resided:

```
case FOURCC('t', 'x', '3', 'g'):
{
    uint32_t type;
    const void *data;
    size_t size = 0;
    if (!mLastTrack->meta->findData(
        kKeyTextFormatData, &type, &data, &size)) {
        size = 0;
    }
    uint8_t *buffer = new (std::nothrow) uint8_t[size + chunk_size];
    if (buffer == NULL) {
        return ERROR_MALFORMED;
    }
    if (size > 0) {
```

```

        return ERROR_MALFORMED;
    }
    if (size > 0) {
        memcpy(buffer, data, size);
    }
    if ((size_t)(mDataSource->readAt(*offset, buffer + size, chunk_size))
        < chunk_size) {
        delete[] buffer;
        buffer = NULL;
        // advance read pointer so we don't end up reading this again
        *offset += chunk_size;
        return ERROR_IO;
    }
    mLastTrack->meta->setData(
        kKeyTextFormatData, 0, buffer, size + chunk_size);
    delete[] buffer;
    *offset += chunk_size;
    break;
}

```

With this big hint, I was able to form a theory:

- If there were *two* 'tx3g' boxes in the file, where the `chunk_size`-s were carefully crafted, we could overflow the expression:

```
uint8_t *buffer = new (std::nothrow) uint8_t[size + chunk_size];
```

- `size` would be the `chunk_size` of the first box, while `chunk_size` would be the size of the second one.
- The problem, as I understood, is that `chunk_size` never exceeds 0xFFFFFFFF (32-bit), so we're not able to produce an overflow.

The danger of not reading and understanding code

At this point Mr. Green pointed out that I had not read the `parseChunk` completely:

```

status_t MPEG4Extractor::parseChunk(off64_t *offset, int depth) {
    ALOGV("entering parseChunk %lld/%d", *offset, depth);
    uint32_t hdr[2];
    if (mDataSource->readAt(*offset, hdr, 8) < 8) {
        return ERROR_IO;
    }
    uint64_t chunk_size = ntohl(hdr[0]);
    uint32_t chunk_type = ntohl(hdr[1]);
    off64_t data_offset = *offset + 8;
    if (chunk_size == 1) {
        if (mDataSource->readAt(*offset + 8, &chunk_size, 8) < 8) {
            return ERROR_IO;
        }
    }
    [...]
}

```

Yikes! I had ignored this code, thinking it was an error condition



In fact, this code enables the specifying of a 64-bit `chunk_size` within an MPEG-4 file! If the 32-bit `chunk_size` is equal to 1, the code reads the next eight (8) bytes as the `chunk_size`! So it is possible to specify the tx3g pointers as 64-bit unsigned integers. From here we have the vulnerability:

- The MPEG-4 meta file contains two (2) tx3g boxes.
 - The first one specifies a normal `chunk_size`. Since it is the first time this chunk type is encountered, and normal buffer is allocated and copied to. This `chunk_size` is stored in the `mLastTrack->meta` vector with a key of `text`.

- The first one specifies a normal `chunk_size`. Since it is the first time this chunk type is encountered, and normal buffer is allocated and copied to. This `chunk_size` is stored in the `mLastTrack->meta` vector with a key of `text`.
- The second box specifies `chunk_size` equals one (1), causing the code to read a 64-bit `chunk_size`. We choose a `chunk_size` that causes `size+chunk_size` to overflow and equal any size we wish.

```
if (!mLastTrack->meta->findData(
    kKeyTextFormatData, &type, &data, &size)) {
    size = 0;
}
uint8_t *buffer = new (std::nothrow) uint8_t[size + chunk_size];
if (buffer == NULL) {
    return ERROR_MALFORMED;
}
```

- Now we come to the `memcpy`:

```
if (size > 0) {
    memcpy(buffer, data, size);
}
```

Since `size` is the previous, normal-sized `chunk_size`, the operation will overflow the buffer by whatever size we want.

- Now we come to the `readAt()` operation:

```
if ((size_t) (mDataSource->readAt(*offset, buffer + size, chunk_size))
    < chunk_size) {
    delete[] buffer;
    buffer = NULL;
    // advance read pointer so we don't end up reading this again
    *offset += chunk_size;
    return ERROR_IO;
}
```

- Since `chunk_size` is ridiculously large, `readAt()` will return a failure return code. Nothing crashes — the code just deletes the buffer and returns!

That's the vulnerability: by crafting the MPEG-4 meta file, we control both the second buffer size and the number of bytes we will overflow the buffer.

✗ Morals learned from StageFright exercise

NOTE! Morals learned from this assignment

Here's what I learned from this assignment:

- Reading about the MPEG-4 file format was important to understand the object-oriented structure of (`chunk_size`, `chunk_TYPE`, `DATA`).
- The use of an MP4 explorer (in my case, MP4 Inspector) was very useful.
 - However, Mr. Green feels that my using it crippled my complete understanding of the MPEG-4 file format — see below).
- I decided to concentrate of the correct functions, specifically `parseChunk`.
- A good methodology is to locate all places that `memcpy` appears in the code. These are good candidates for buffer overflows.
- You **MUST** read the code carefully and understand the purpose of every line. Do not assume code to be of little value (as Mr. Green says, "the code never lies").
 - In my case, I failed to understand the importance of `if (chunk_size == 1)`. This was super critical to realizing that it is possible to specify 64-bit chunk sizes.

Mr. Green says, “the code never lies”).

- In my case, I failed to understand the importance of `if (chunk_size == 1)`. This was super critical to realizing that it is possible to specify 64-bit chunk sizes.
- Mr. Green points out that code with buffer, offsets, and user-crafted input is just asking to be attacked.

Project Zero: StageFright Vulnerability and Exploit

For the most complete exposition of this vulnerability, and to see how it is exploited, see [Project Zero: Stagefrightened?](#)